

HEX0N: A Simple Set of Rules for Evaluating Complex Reasoning in AI Agents and Language Models

Georgios Tritsaridis*

Abstract

Classic games used to test artificial intelligence models come with vast amounts of strategy text in pre-training data, so strong play may partly reflect memorization. We present HEX0N, a placement game with a single chance element. The game can be thought of as a small abstract economy: a scarce endowment, a free input, a production step, rival land and a property rule that says whose inputs may be used, so play can be scored with quantitative indicators as well as results. Self-play agents, each trained under a different combination of rules, settle into recognizably different regimes. An agent moved to the other property regime scores 0–15% against the one trained there. Moreover, a pilot with language models shows them following the rules but playing a production race, which the trained agents learned to avoid.

1 Introduction

Static benchmarks for language models are vulnerable to contamination when test items appear in training corpora [1]. Games are an attractive complement: outcomes are objective, difficulty scales with the opponent, and new positions arise in every game. This motivated platforms such as Kaggle Game Arena [2], where models play games against each other and are ranked on a public leaderboard. Relatedly, OpenSpiel [3] provides a framework for studying reinforcement learning, search, and strategic decision-making across a wide range of games. Most of these platforms rely on well-known games whose openings and annotated records are abundant online, so a model may play well by recalling human strategy rather than by reasoning about the rules it is given. HEX0N is a newly designed placement game, intended to reduce this source of contamination while retaining the strategic depth needed to distinguish models of different capabilities.

HEX0N’s rules simultaneously define a minimal economy: winning requires reasoning such as whether to supply an input the rival can appropriate, when to hold inventory, and when to exercise an option to produce. Economic and social behavior emerging from learning has been studied in sequential social dilemmas [4] and simulated economies [5], and language models have been tested as economic agents [6]. For instance, Chen et al. [6] find that GPT exhibits largely rational economic choices, outperforming human subjects in consistency with utility maximization, although its decisions remain sensitive to how choices are framed.

Our main contribution is to bring these threads together in an abstract setting. We train four agents by self-play, one for each version of two rules, rate strength against random play and tree search, and evaluate their games and those of language models with the same indicators. For example, switching one property rule, whether a player may use the opponent’s pieces, turns trained

*Generative AI tools were used to assist with the analysis and its summarization. The author reviewed and takes responsibility for the final content.

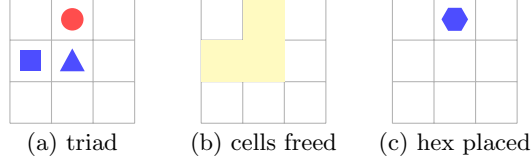


Figure 1: A triad. Blue’s triangle touches Blue’s square and Red’s circle, so all three are removed and Blue places a permanent hex on one of the freed cells.

play from parallel production into contest, and the same indicators show what language models miss: they produce eagerly but do not guard against appropriation.

Section 2 gives the rules, the two rule switches that define four institutional regimes, the reference scale and the language-model protocol. Section 3 rates the trained agent of the canonical rules against that scale, describes the regime each trained agent settles into and tests whether its skill survives a change of rules, and then applies the same indicators to a pilot with two language models. Section 4 outlines the planned evaluation and the open questions.

2 Game and Methods

2.1 Game Rules

HEX0N is played on an $S \times S$ grid; here we limit ourselves to $S = 5, 7$. Two cells are adjacent if they touch orthogonally or diagonally. In the canonical version, two players, Blue and Red, alternate, and on each turn the mover places one piece on an empty cell:

- **Square:** anywhere, from a supply of $\text{round}(S^2/3)$ per player (8 on 5×5).
- **Circle:** next to at least one square of either color; unlimited supply.
- **Triangle:** next to a circle and a square of any colors, at least one of the two the mover’s. The triangle, that circle and that square are removed (a *triad*); if several circle–square pairs qualify, one is drawn uniformly at random. In the same turn, the mover places a permanent **hex** on one of the three freed cells.

The game proceeds as follows:

- **Passing:** a player who cannot place anything passes.
- **End:** the game ends when the board is full or neither player can place.
- **Winner:** more hexes wins; ties go to the player with the larger connected group of hexes, and otherwise the game is drawn.

Figure 1 shows a triad. On the 5×5 board, the game has 101 possible actions: placing a square, circle, or triangle in any of the 25 cells, placing a hex in any cell following a triad, and passing, with only legal actions available at each decision.

Variations. Two rules can be switched without changing anything else. The *property rule* (“opponent’s piece”, denoted as **opp**) lets a triad use one piece of the opponent; with it off (**noopp**), both pieces must be the mover’s, so inputs are secure. The *tie-break* (**tie**) is a secondary prize for agglomeration; with it off (**notie**), equal hexes is a draw. The four resulting rule sets (or institutional regimes) are therefore **tie_opp** (the canonical game), **notie_opp**, **tie_noopp**, and **notie_noopp**.

Match	Games	W–D–L	Score	95% CI	Agent	Elo	95% CI
mcts4 vs random	40	31–0–9	0.78	[0.62, 0.88]	random	0	(anchor)
mcts16 vs random	40	36–1–3	0.91	[0.79, 0.97]	mcts4	230	[131, 336]
mcts16 vs mcts4	40	26–2–12	0.68	[0.52, 0.80]	mcts16	371	[263, 507]
mcts64 vs mcts4	40	39–0–1	0.97	[0.87, 1.00]	mcts64	833	[671, 1141]
mcts64 vs mcts16	40	37–0–3	0.93	[0.80, 0.97]	mcts256	1147	[962, 1487]
mcts256 vs mcts16	40	40–0–0	1.00	[0.91, 1.00]	mcts1024	1390	[1210, 1785]
mcts256 vs mcts64	40	34–0–6	0.85	[0.71, 0.93]	tie_opp	2088	[1851, 2514]
mcts1024 vs mcts64	30	29–0–1	0.97	[0.83, 0.99]			
mcts1024 vs mcts256	30	24–0–6	0.80	[0.63, 0.90]			
tie_opp vs mcts256	30	30–0–0	1.00	[0.89, 1.00]			
tie_opp vs mcts1024	32	31–1–0	0.98	[0.87, 1.00]			

Table 1: The reference scale. Left: matches, score of the first-named (stronger) agent with Wilson 95% intervals. Right: Bradley–Terry ratings on the Elo scale, random = 0, 95% bootstrap intervals. **tie_opp** plays its best move.

Statistic	5×5		7×7	
	random	tie_opp _{T=0.20}	random	tie_opp _{T=0.20}
Games	1000	500	500	200
Plies (mean)	43.5	39.0	88.3	90.8
Plies (s.d.)	7.5	7.9	11.9	14.1
Legal moves per decision	18.8	17.2	36.6	27.7
Triads per game	6.6	4.9	13.7	14.7
Triads with a random pair	65%	24%	70%	39%
Passes per game	0.08	0.06	0.09	0.01
First player wins	50%	80%	53%	59%
Draws	9.5%	1.4%	3.8%	0.0%
Decided by tie-break	11%	3.8%	9%	1.5%

Table 2: Descriptive statistics for self-play. “Triads with a random pair” indicates that more than one circle-square pair qualified.

2.2 The reference scale

A reference scale is established so that agents can be evaluated:

- **random:** Plays uniformly at random and anchors the rating scale at 0.
- **mctsN:** Monte Carlo Tree Search using Upper Confidence Bounds applied to Trees (UCT) [7] with N simulations per move, random rollouts, and exploration constant $c = 2$, as implemented in OpenSpiel.
- **Self-play agents:** Neural networks trained through self-play under the four rule sets, selecting moves either deterministically (denoted by the rule-set name, e.g., **tie_opp**) or by sampling from their policy at a specified temperature T (indicated by an additional suffix, e.g., **tie_opp**_{T=0.25} for $T = 0.25$).

Ratings are estimated using the Bradley–Terry model [8], fitted via minorization–maximization (MM) iterations [9] (draws counted as half a win). Estimated strengths are converted to the Elo scale, and 95% confidence intervals are obtained from 300 bootstrap resamples of games. To ensure fairness, matches alternate the starting player, and each game uses an independent random seed.

Indicator (per game, self-play)	tie_opp $T=0.25$	notie_opp $T=0.25$	tie_noopp $T=0.25$	notie_noopp $T=0.25$
Hexes made, both players	5.2	10.0	9.4	8.8
Games level on hexes	4%	14%	72%	92%
Moves a circle survives before use	1.6	11.8	4.6	4.3
Squares never converted, of 8	5.4	2.9	3.3	3.6
Takes a triad when one is available	80%	50%	91%	72%
New hex touches an own hex	33%	51%	75%	45%
Lead changes	0.3	1.2	1.4	1.1

Table 3: Four regimes. Each agent plays itself under its own rules (50 games, no search, temperature 0.25).

2.3 Language model setup

For benchmarking of language models, each decision is a fresh request containing the rules, an ASCII board, a one-line position code in the style of chess FEN notation (for example `5/1S3/2c2/5/5 b main 7 8 0 0 -`: the board row by row, the side to move, the stage, squares left, hexes and cells awaiting a hex), the scores, the square supply, the move history and, by default, the list of legal moves, written `S b2, C c3, T d4, H a1`. No tools are offered. The model must end with `Final Answer: <move>`.

An illegal or unreadable answer receives rule-based feedback and another try, three attempts in all, after which the model forfeits. Decisions with a single legal move are played automatically. A second prompt version, not used in the pilot below, states the rules in three equivalent wordings, rotated every two games so that each is played from both seats, to test sensitivity to phrasing.

3 Results and Discussion

Each self-play agent is a residual convolutional neural network (CNN; 64 channels, 4 blocks, about 0.3M parameters) with a policy head over the 101 actions and a win/draw/loss value head, trained by self-play with Gumbel AlphaZero search [10, 11]: 32 simulations per decision, the triad pair as an enumerated chance node, the eight board symmetries, and only the final result as value target. We trained one agent per combination of the two rules: **tie_opp** (both on in the canonical version), **notie_opp**, **tie_noopp** and **notie_noopp**, each for about 500,000 decisions. We then recorded 16 matches of 50 games each: each agent against itself and each pair under each pair’s rule sets, the network alone without search, sampling at temperature 0.25 (**tie_opp** $T=0.25$ and so on). **tie_opp**, playing its best move, is also the trained reference in Section 2.2.

Table 1 shows a connected, well-separated scale. Random, although not hopeless, scores 0.22 against mcts4. Each fourfold increase in simulations is worth about 140 to 460 Elo, and mcts1024 sits about 1,400 above random.

The trained agent **tie_opp**, playing its best move with a single network evaluation per move, won all 30 games against mcts256 and 31 of 32 against mcts1024, with one draw. Its rating of about 2,100 rests on that single draw and is only a rough figure; a higher rung (e.g., mcts4096) is needed to rate it well. The game rewards understanding over brute force: random rollouts are poor estimates here because random play wastes squares and hands triads to the opponent.

Table 2 describes the game under random play and under self-play by **tie_opp** $T=0.20$. On 5×5 , games last about 40–45 plies with 17–19 legal moves per decision. Trained play makes fewer triads than random play (4.9 vs. 6.6 a game) and avoids random pairs far more often (24% vs. 65% of triads), so controlling the chance element is part of skill. Tempo matters: the first player wins 80% of trained self-play games on 5×5 and 59% on 7×7 , so matches must alternate seats. The network, trained only on 5×5 , plays complete games on 7×7 , a harder variant with the same rules and

Agent	Regime	Representative game	Economic analogy
tie_opp $T=0.25$	Standoff	Lays squares first; first circle after move 10. 84% of used circles taken on the next move; 5.4 of 8 squares never used. Lead changes 0.3 times a game; the last mover won 43 of 49 decisive games.	War of attrition
notie_opp $T=0.25$	Raider and trader	Stocks circles already from move 2 and holds them 11.8 moves; 48% of its hexes take the opponent’s square. Highest output (10 hexes a game) and most lead changes. Takes an available triad only half the time.	Inventory as stock; Option value
tie_noopp $T=0.25$	Cluster builder	Opens in the center and produces alone, just in time. 75% of new hexes touch its own; its largest group holds 4.5 of its 4.7 hexes.	Agglomeration
notie_noopp $T=0.25$	Matcher	Produces alone and ignores location. A third of its hexes answer the opponent’s hex on the next move; 92% of games end level.	Parity without exchange; tit-for-tat

Table 4: The four institutional regimes (self-play, 50 games each).

Home agent (its rules)	vs tie_opp $T=0.25$	vs notie_opp $T=0.25$	vs tie_noopp $T=0.25$	vs notie_noopp $T=0.25$
tie_opp $T=0.25$	–	0.40	0.98	0.96
notie_opp $T=0.25$	0.64	–	0.96	1.00
tie_noopp $T=0.25$	0.97	0.97	–	0.78
notie_noopp $T=0.25$	0.91	0.85	0.55	–

Table 5: Agents away from home. Score of the home agent (wins plus half the draws, 50 games) under its own rules.

twice the length.

Four institutional regimes. Each agent settled into its own way of spending squares, circles and turns. Table 3 gives the indicators, and Table 4 pairs each regime with the economic idea. With inputs contestable, **tie_opp** $T=0.25$ exhibits behavior consistent with a hold-up mechanism: 84% of the circles that get used are appropriated by the opponent on the next move, discouraging input provision and leaving 5.4 of 8 squares unused. With inputs secure, the agents produce in parallel and in step, and most games end level (72–92%).

We find therefore that skill is institution-specific. To explore this property further, we played each agent under another agent’s rules. Table 5 shows how a visitor from the other property regime makes only legal moves but scores 0–15%, although across the tie-break alone skill transfers far better (home scores 0.40–0.78). For example, **tie_opp** $T=0.25$ and **notie_opp** $T=0.25$ face seemingly the same incentives, since the tie-break settles few of their games, yet they open in opposite ways and **notie_opp** $T=0.25$ wins their meetings 58–60%.

3.1 Language models

In a pilot, two models played each other on 5×5 , two games with the first move alternated, the legal-move list shown and prompt version 1: Claude Haiku 4.5 (**claude-haiku-4-5-20251001**, extended thinking with a 4,000-token budget) and GPT-5.6 Luna (**gpt-5.6-luna**, reasoning effort medium), both with a 16,000-token ceiling. Two games suggest preferred strategy, but they cannot be used for a robust ranking. Luna won both, 8–7 and 8–4 in hexes (Table 6).

	Haiku 4.5	GPT-5.6 Luna
Result (2 games)	0 wins	2 wins
Hexes per game	5.5	8.0
Decisions / answers	56 / 56	62 / 64
Illegal answers, forfeits	0, 0	2 (3.1%), 0
Tokens in / out per decision (thinking incl.)	822 / 2,240	748 / 2,215
Seconds per decision	19.3	24.7

Table 6: Pilot: Claude Haiku 4.5 against GPT-5.6 Luna, two games.

Indicator (per player and game)	random	tie_opp_{T=0.25}	Haiku 4.5	GPT-5.6 Luna
Games (seats counted separately)	1000	100	2	2
Hexes made	3.3	2.6	5.5	8.0
Takes a triad when one is available	22%	80%	92%	94%
Exposing moves (hand the opponent a triad)	0.8	3.3	5.5	4.0
of which a safe move existed	91%	16%	73%	100%
Moves a circle survives before use	7.6	1.6	1.6	1.7
Own hexes using an opponent’s piece	73%	91%	64%	81%
Squares never converted, of 8	4.7	5.4	2.0	0.5

Table 7: Descriptive statistics for language models. See also Table 3.

Both models stayed within the rules. Haiku gave no illegal answer; Luna gave two, each corrected on the next attempt: once it answered with two moves at once (**T b5**, **H a5**), and once it named a triangle absent from the list it was given. Table 7 scores the pilot games with the indicators of Section 3. The models play a production race. They take 92–94% of available triads, more than any trained agent, convert almost all their squares (Luna left 0.5 of 8 unused, against 5.4 for **tie_opp_{T=0.25}**), and produce more hexes per game than **tie_opp_{T=0.25}** or random that plays under the same mechanics. The other side of the ledger is appropriation. Each model handed the opponent a triad 4–5.5 times a game; **tie_opp_{T=0.25}** exposes less often, and five times in six only when nothing safe is left. In strategic terms, the models reason about their own production but not about the rival’s: they supply inputs the other side converts at once, the hold-up that **tie_opp_{T=0.25}** learned to avoid. Between two such players the game becomes a raiding economy like **notie_opp_{T=0.25}**’s, but by accident rather than by choice, and the result turns on who exposes less.

4 Conclusions

HEXON is a new board game, so frontier artificial intelligence models have no strategy writing about it to rely on. Its rules form a small abstract economy: scarce inputs, production, land, and a rule on whether you may use the opponent’s pieces. We trained agents by self-play and rated play against random play and Monte Carlo tree search. We then scored the games and a pilot between two language models with the same indicators.

The property rule alone changes how the game is played, from contest when inputs can be taken to parallel production when they cannot, and skill does not transfer between the two (0–15% scores). Language models follow the rules but play a production race, often handing the opponent a triad when a safe move exists.

The setup leaves open questions that further simulation can settle: whether the four regimes reappear with more seeds and longer training; whether rewarding own hexes rather than winning makes mutually beneficial exchange appear under contestable inputs; what happens under graded property rights, such as a cost in turns or squares for using an opponent’s piece; and whether a

language model, which reads the rules, switches regime when they change while a trained network carries one habit across.

Extending this pilot to more advanced models at more games per pairing, is a logical next step. Running the second prompt version, with its three equivalent wordings, would show whether results depend on phrasing.

References

- [1] Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 10776–10787, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.722. URL <https://aclanthology.org/2023.findings-emnlp.722/>.
- [2] Kaggle and Google DeepMind. Kaggle Game Arena. Online benchmarking platform, 2025. URL <https://www.kaggle.com/game-arena>. Official launch announcement: <https://www.kaggle.com/blog/introducing-game-arena/>.
- [3] Marc Lanctot, Edward Lockhart, Jean-Baptiste Lespiau, Vinicius Zambaldi, Satyaki Upadhyay, Julien Pérolat, Sriram Srinivasan, Finbarr Timbers, Karl Tuyls, Shayegan Omidshafiei, Daniel Hennes, Dustin Morrill, Paul Muller, Timo Ewalds, Ryan Faulkner, János Kramár, Bart De Vylder, Brennan Saeta, James Bradbury, David Ding, Sebastian Borgeaud, Matthew Lai, Julian Schrittwieser, Thomas Anthony, Edward Hughes, Ivo Danihelka, and Jonah Ryan-Davis. OpenSpiel: A framework for reinforcement learning in games. *arXiv preprint arXiv:1908.09453*, 2019. URL <https://arxiv.org/abs/1908.09453>.
- [4] Joel Z. Leibo, Vinicius Zambaldi, Marc Lanctot, Janusz Marecki, and Thore Graepel. Multi-agent reinforcement learning in sequential social dilemmas. In *Proceedings of the 16th International Conference on Autonomous Agents and Multiagent Systems*, pages 464–473, 2017. URL <https://dl.acm.org/doi/10.5555/3091125.3091194>.
- [5] Stephan Zheng, Alexander Trott, Sunil Srinivasa, David C. Parkes, and Richard Socher. The AI economist: Taxation policy design via two-level deep multiagent reinforcement learning. *Science Advances*, 8(18): eabk2607, 2022. doi: 10.1126/sciadv.abk2607.
- [6] Yiting Chen, Tracy Xiao Liu, You Shan, and Songfa Zhong. The emergence of economic rationality of GPT. *Proceedings of the National Academy of Sciences*, 120(51):e2316205120, 2023. doi: 10.1073/pnas.2316205120.
- [7] Levente Kocsis and Csaba Szepesvári. Bandit based Monte-Carlo planning. In *Machine Learning: ECML 2006*, pages 282–293. Springer, 2006. doi: 10.1007/11871842_29.
- [8] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. doi: 10.2307/2334029.
- [9] David R. Hunter. MM algorithms for generalized Bradley–Terry models. *The Annals of Statistics*, 32(1): 384–406, 2004. doi: 10.1214/aos/1079120141.
- [10] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018. doi: 10.1126/science.aar6404.
- [11] Ivo Danihelka, Arthur Guez, Julian Schrittwieser, and David Silver. Policy improvement by planning with Gumbel. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=bERaNdogn0>.

A Example transcript for Haiku

Agent: Claude Haiku 4.5

Position: 1sC2/1ShcC/1shS1/1HHh1/2cS1 b main 3 3 2 3 -

Prompt:

Current position (you are Blue):

```
  a b c d e
1 . s C . .
2 . S h c C
3 . s h S .
4 . H H h .
5 . . c S .
```

Position code: 1sC2/1ShcC/1shS1/1HHh1/2cS1 b main 3 3 2 3 -

Hexes - Blue: 2, Red: 3

Largest hex group - Blue: 2, Red: 3

Squares left - Blue: 3, Red: 3

It is your turn to place a piece.

Legal moves: S a1, S d1, S e1, S a2, S a3, S e3,

S a4, S e4, S a5, S b5, S e5, C a1, C a2,

C a3, C e3, C a4, C e4, C e5, T e3

Model response:

Looking at this position, I need to analyze the current board state and available moves.

****Current Analysis:****

- Blue: 2 hexes, 3 squares remaining
- Red: 3 hexes, 3 squares remaining
- The board is getting filled, so tempo matters

****Evaluating T e3:****

Position e3 is adjacent to:

- e2 (Blue C - circle)
- d3 (Blue S - square)
- d2 (Red c - circle)

This gives qualifying circle+square pairs:

- (e2, d3): both Blue's pieces
- (d2, d3): one Red circle + Blue square

Either way, placing a triangle at e3 will:

1. Remove one of these pairs + the triangle
2. Free up three cells (e2, d3, e3)
3. Allow me to immediately place a hex on one of those cells

****Why this move:****

- Guarantees gaining a hex (+1), bringing us to 3 hexes
- In either scenario, d3 becomes available and I'd place the hex there,

consolidating control in the center-right area

- Removes one of Blue's circles but eliminates either one of Red's circles or helps us lose a square strategically
- Builds better hex connectivity (hexes at b4, c4, and the new one at d3)

This is a strong offensive move that improves my hex count and board position.

Final Answer: T e3

Observation. *The model correctly identifies the two eligible circle-square pairs associated with placing a triangle at e3. However, it subsequently assumes that e2, d3, and e3 are removed in both cases. If the pair (d2,d3) is selected, the removed cells are instead d2, d3, and e3. Thus, although the selected move is legal and the immediate production opportunity is correctly recognized, the explanation conflates the two possible stochastic outcomes. Its claims about the resulting board configuration and positional advantage are therefore only partially supported.*